

1. What is C Language?

C is a **general-purpose programming language** used for system programming and application development.

It is:

- Fast
- Efficient
- Close to hardware

👉 Used in:

- Operating systems
- Embedded systems
- Software development

2. Structure of C Program

```
#include <stdio.h>

int main() {
    printf("Hello World");
    return 0;
}
```

Explanation:

- `#include <stdio.h>` → includes standard input/output library
- `int main()` → main function (starting point)

- `printf()` → prints output
- `return 0;` → ends program

Use:

Basic program structure

3. Keywords

Keywords are reserved words.

Examples:

`int`, `float`, `if`, `else`, `return`, `while`

Use:

Cannot be used as variable names

4. Variables

```
int a = 10;
```

```
float b = 5.5;
```

```
char c = 'A';
```

Explanation:

- `int` → integer
- `float` → decimal
- `char` → character

Use:

Store data

5. Data Types

Type	Example
int	10
float	5.5
char	'A'
double	10.123

Use:

Define type of data

6. Constants

```
const int x = 10;
```

Use:

Value cannot be changed

7. Input and Output

```
int a;  
scanf("%d", &a);  
printf("%d", a);
```

Explanation:

- scanf() → takes input
- printf() → prints output

- `&a` → address of variable

Use:

User interaction

8. Operators

```
int sum = a + b;
```

Types:

- Arithmetic → `+` `-` `*` `/` `%`
- Relational → `==` `!=` `>` `<`
- Logical → `&&` `||` `!`

Use:

Perform operations

9. Conditional Statements

```
if(a > 10) {  
    printf("Greater");  
} else {  
    printf("Smaller");  
}
```

Use:

Decision making

10. Switch Statement

```
switch(a) {  
    case 1: printf("One"); break;  
    default: printf("Other");  
}
```

Use:

Multiple conditions

11. Loops

For Loop

```
for(int i=0; i<5; i++) {  
    printf("%d", i);  
}
```

While Loop

```
while(i < 5) {  
    i++;  
}
```

Do-While

```
do {  
    i++;  
} while(i < 5);
```

Use:

Repeat code

12. Arrays

```
int arr[3] = {1,2,3};
```

Use:

Store multiple values

13. Strings

```
char str[] = "Hello";
```

Use:

Store text

14. Functions

```
void greet() {  
    printf("Hello");  
}
```

Use:

Reusable code

15. Function with Return

```
int add(int a, int b) {  
    return a + b;  
}
```

Use:

Return value

16. Pointers (Important)

```
int a = 10;  
int *p = &a;
```

Explanation:

- `*p` → pointer variable
- `&a` → address

Use:

Memory management

17. Structures

```
struct student {  
    int id;  
    char name[20];  
};
```

Use:

Group data

18. Union

```
union data {  
    int a;  
    float b;  
};
```

Use:

Save memory

19. File Handling

```
FILE *fp;  
fp = fopen("file.txt", "r");
```

Use:

Read/write files

20. Dynamic Memory Allocation

```
int *p = (int*) malloc(sizeof(int));
```

Use:

Allocate memory at runtime

21. Preprocessor Directives

```
#define PI 3.14  
#include <stdio.h>
```

Use:

Define constants and include files

22. Recursion

```
int fact(int n) {  
    if(n == 0) return 1;  
    return n * fact(n-1);  
}
```

Use:

Solve problems using function calls

23. Storage Classes

- auto
- static
- extern
- register

Use:

Define scope and lifetime

24. Bitwise Operators

```
a & b;  
a | b;
```

Use:

Bit-level operations

25. Type Casting

```
float x = (float)10/3;
```

Use:

Convert data types

26. Command Line Arguments

```
int main(int argc, char *argv[])
```

Use:

Pass input via terminal